Project Report: Implementing and Evaluating Two Strategies for the Traveling Salesman Problem

Matei Havarneanu Petru Ghionea

April 23, 2026

Abstract

This project studies the Traveling Salesman Problem (TSP) through two strategies: the classical Lin-Kernighan-Helsgaun heuristic (LKH), used as a strong baseline solver, and a hybrid learning-based strategy inspired by NeuroLKH. The implemented system uses a Sparse Graph Network (SGN) to predict edge scores and node penalties, then passes this information to LKH in order to guide candidate-set construction and distance transformation during local search. The report explains the design decisions behind the implementation, including the sparse directed graph representation, the multi-task training objective, the meaning of the SGN outputs, and the modular project structure. It also discusses the benchmark experiments, the observed behavior of training, and the main limitations of the implementation relative to the original paper.

1 Problem Statement

The Traveling Salesman Problem (TSP) asks for a minimum-cost Hamiltonian tour that visits each city exactly once and returns to the starting city. TSP is one of the best-known NP-hard combinatorial optimization problems, and it is a standard benchmark for both exact and heuristic algorithms.

The project requirement was to implement at least two strategies for the chosen problem, motivate the design choices, and test the strategies on benchmark instances. For this reason, the project was organized around the following two strategies:

1. **Classical LKH baseline.** The Lin-Kernighan-Helsgaun heuristic is used as the reference strategy.
2. **Hybrid NeuroLKH-style strategy.** A Sparse Graph Network predicts edge scores and node penalties, and these outputs are used to guide the LKH search procedure.

The project follows the ideas from the paper *NeuroLKH: Combining Deep Learning Model with Lin-Kernighan-Helsgaun Heuristic for Solving the Traveling Salesman Problem* (NeurIPS 2021), but it should be understood as an implementation inspired by the paper rather than a perfect reproduction.

2 Overview of the Two Strategies

2.1 Strategy 1: Classical LKH

LKH is a strong local-search heuristic based on variable-depth λ-opt moves. It constructs a candidate set of promising edges and uses transformed edge distances to guide the search. In the original LKH algorithm, candidate sets are derived from hand-crafted rules based on minimum 1-trees and sensitivity analysis, while node penalties are optimized by subgradient optimization.

In this project, LKH is not reimplemented from scratch in Python. Instead, the official executable is called as an external solver. This is still a valid strategy in the project because it is the baseline method against which the hybrid method is compared. LKH is used in two places: to generate labels for the training dataset and to compute final solutions during inference and benchmarking.

2.2 Strategy 2: Hybrid NeuroLKH

The second strategy combines a neural model with the LKH solver. The neural component is a Sparse Graph Network (SGN) that receives a sparse directed graph representation of the TSP instance and predicts two outputs: edge scores $\beta_{i,j}$, used to build a better candidate set, and node penalties π_i, used to transform the original edge distances.

The resulting workflow is: convert a TSP instance into a sparse directed graph, run SGN inference and obtain edge scores and node penalties, transform the edge distances using the predicted node penalties, create a candidate edge set from the predicted edge scores, and pass the transformed instance and candidate set to LKH.

3 Design Choices and Motivation

3.1 Sparse Directed Graph Representation

A central design decision was to represent each TSP instance as a sparse directed graph. For every node, only the γ nearest outgoing edges are kept. In the implementation, $\gamma = 20$ was used.

This choice was motivated by the paper and has several advantages: it drastically reduces the number of edges processed by the neural model, it keeps the most relevant local geometric information, and it makes training feasible on larger instances than a dense graph would allow. The graph is directed, which means that edge (i, j) and edge (j, i) are stored separately. This is important because the model explicitly learns separate embeddings for opposite directions.

3.2 Neural Architecture

The SGN contains a node encoder that projects 2D coordinates into a hidden feature space, an edge encoder that projects edge distances into the same feature space, 30 sparse graph convolutional layers, an edge decoder that predicts edge scores, and a node decoder that predicts node penalties. The model dimension was set to $D = 128$, and the number of convolutional layers was set to $L = 30$, following the same values reported in the article.

3.3 Why Two Outputs?

The model predicts both edge scores and node penalties because the paper argues that both are critical to improving LKH. Their roles can be explained more explicitly as follows:

- **Edge scores β.** For each city, the model looks at all outgoing edges that remain in the sparse graph and produces a probability distribution over them. A high score means that the neural network considers that edge more likely to belong to a good or near-optimal tour. The scores of all outgoing edges from the same city sum to 1, so they can be interpreted as relative priorities. These scores are then used to build the candidate set passed to LKH.

- **Node penalties π.** For each city, the model predicts a scalar value between -10 and $+10$. These values are used to reshape the original edge distances before the LKH search. A negative penalty makes edges incident to that city effectively shorter, so LKH will tend to prefer them. A positive penalty makes such edges longer, so LKH will tend to avoid them. In this way, the model does not construct the tour directly, but biases the search performed by LKH.

Using both outputs was a natural design choice because it keeps the implementation close to the algorithmic idea of NeuroLKH: one output improves the candidate set, while the other modifies the search landscape through distance transformation.

3.4 Training Objective

The project uses a multi-task loss, combining a supervised edge loss L_β and an unsupervised node-penalty loss L_π. The total loss is

$$L = L_\beta + \eta_\pi L_\pi,$$

where $\eta_\pi = 1$. The edge loss encourages large scores for edges that belong to the reference tour and small scores otherwise. The node loss follows the idea from the paper: it pushes the degrees of the minimum 1-tree toward 2 by adjusting node penalties. This choice is appropriate because it mirrors the two ways in which NeuroLKH helps LKH: candidate-set improvement and edge-distance transformation.

3.5 Data Structures

The implementation uses `numpy.ndarray` for coordinates and distance-related processing, `torch.Tensor` for neural-network computation and GPU acceleration, Python lists for edge lists and candidate sets, and serialized pickle files for training data. These choices are practical and appropriate for the project: PyTorch tensors are necessary for the neural model and CUDA execution, NumPy and SciPy simplify auxiliary graph computations, Python lists keep the implementation readable, and pickle offers a simple way to store pre-generated training instances.

3.6 Modular Design

The project was organized into separate modules: `data_generator.py` for dataset generation, `sgn.py` for the Sparse Graph Network, `train.py` for training, `solver.py` for inference and interaction with the LKH executable, and `benchmark.py` and `run.py` for evaluation. This modular design was chosen because it improves clarity, separates responsibilities, and makes experimentation easier.

3.7 Project Structure

The implementation also follows a clear directory structure, which makes the workflow easier to understand and reproduce:

```
neurolkh/
|-- benchmark_instances/
|   |-- a280.tsp
|   |-- ali535.tsp
|   |-- att48.tsp
|   |-- ...
|   |-- ulysses22.tsp
|   |-- usa13509.tsp
|   |-- vm1084.tsp
|   `-- vm1748.tsp
|-- checkpoints/
|   `-- neurolkh_best.pt
|-- LKH-3.0.14/
|   `-- LKH-3.exe
|-- training_data/
```

```
|   `-- training_data.pkl
|-- benchmark.py
|-- data_generator.py
|-- run.py
|-- sgn.py
|-- solution.txt
|-- solver.py
`-- train.py
```

This structure reflects the main stages of the project: benchmark instances are stored separately, training data and trained checkpoints are kept in dedicated folders, the LKH executable is included as an external dependency, and the Python modules are separated according to their responsibility in the pipeline.

4 Dataset and Training Procedure

The training file `training_data.pkl` contains a list of TSP instances. Each training sample stores coordinates of the nodes, a reference tour, the corresponding tour distance, the instance size, and the generation type (uniform or clustered).

The instances were generated synthetically, using either uniform sampling in the unit square or clustered sampling. For each generated instance, LKH was run to obtain the reference tour and distance.

This is an important difference from the paper: the original article uses Concorde to obtain optimal edges for supervised training on TSP, while in this project the training labels were generated with LKH. Therefore, the implementation follows the main NeuroLKH idea, but the supervision source is different.

Training was performed with the following main hyperparameters: hidden dimension $D = 128$, number of layers $L = 30$, sparsity parameter $\gamma = 20$, learning rate 10^{-4}, and 16 training epochs.

In practice, the training set contained 11,599 instances and one full epoch took multiple hours on consumer hardware. This is expected because the implementation processes one instance at a time and the 1-tree degree computation uses a mixture of PyTorch, NumPy, and SciPy.

5 Benchmarking and Experimental Results

The benchmark pipeline loads TSPLIB-style instances and runs the trained model together with LKH. During inference, the solver performs the following steps: run SGN on the input coordinates, compute transformed distances using the predicted node penalties, build candidate sets from the predicted edge scores, and call LKH as the final search engine.

5.1 Observed Results

The updated benchmark processed **62 TSPLIB instances** successfully across multiple edge-weight types, including EUC_2D, ATT, and GEO. For each processed instance, the hybrid solver was run 10 times and the best value was compared with the known optimum provided by TSPLIB. The overall picture is mixed but informative: the method solved **21/62** tested instances to the known optimum, achieved an additional **15/62** within 1% of optimum, and another **6/62** within 10%. The remaining cases showed larger gaps, which indicates that the learned guidance is useful but not uniformly robust across all TSPLIB structures.

Table 1 summarizes these aggregate results, while Table 2 gives representative instances from three regimes: exact matches, near-optimal solutions, and difficult cases.

Category	Count	Share
Exact optimum found	21	33.9%
Gap $\leq 1\%$ (excluding exact)	15	24.2%
Gap in $(1\%, 10\%]$	6	9.7%
Gap $> 10\%$	20	32.3%
Total processed instances	62	100.0%

Table 1: Aggregate benchmark outcome over the 62 processed TSPLIB instances.

Instance	Type	Best obtained	Optimum	Gap (%)
att48	ATT	10628	10628	0.00
berlin52	EUC_2D	7542	7542	0.00
kroA100	EUC_2D	21282	21282	0.00
lin105	EUC_2D	14379	14379	0.00
att532	ATT	27705	27686	0.07
d657	EUC_2D	49168	48912	0.52
p654	EUC_2D	34769	34643	0.36
a280	EUC_2D	3780	2579	46.57
ali535	GEO	396518	202339	95.97
gr202	GEO	90904	40160	126.35

Table 2: Representative benchmark results obtained with the hybrid solver. Exact matches appear on several classical Euclidean and ATT instances, while GEO-type instances and some harder benchmarks remain challenging. The gap is computed as $(\text{best} - \text{optimum})/\text{optimum} \times 100$.

These results show that the implementation can recover the exact optimum on a substantial subset of classical TSPLIB instances, especially among several Euclidean and ATT problems such as `att48`, `berlin52`, `kroA100`, and `lin105`. It also remains very close to optimal on instances such as `att532`, `d657`, and `p654`. However, performance degrades considerably on a number of more difficult or differently parameterized problems, especially some GEO instances such as `ali535` and `gr202`. Overall, the benchmark suggests that the learned guidance is often useful, sometimes extremely accurate, but still sensitive to instance structure and distribution shift.

Training also behaved consistently: the dataset was loaded correctly, all training steps were executed with zero skipped samples, and the loss remained stable and improved slightly over epochs.

The small improvement of the total loss is not necessarily a sign of failure. In this implementation, the node-penalty loss dominates numerically and remains almost constant, while the edge loss improves more slowly. As a result, the total loss changes only slightly, even though the model is still learning.

5.2 Improved TSPLIB Support

The benchmark pipeline was extended to handle TSPLIB instances more robustly. Instead of relying only on a hand-written parser for coordinate sections, the evaluation code was modified to use `tsplib95` for loading benchmark instances and to compute final tour costs using the exact edge-weight function provided by the TSPLIB problem object when available.

This improvement means that the updated benchmark is no longer limited only to EUC_2D cases. It can also evaluate instances with other edge-weight types, such as ATT and GEO, much more faithfully than the original parser. In practice, this is reflected by successful runs on instances such as `att48`, `att532`, and `ali535`.

The remaining limitation is that the neural component still requires node coordinates as input. Therefore, benchmark results should be interpreted as valid for TSPLIB instances that both provide coordinates for the SGN and have edge-weight types supported by the updated evaluation pipeline. In practice, this includes not only EUC_2D, but also instances such as ATT and GEO. Instances without usable coordinates cannot yet be processed by the learned component.

6 Discussion

Overall, the implementation successfully demonstrates the main idea of NeuroLKH: deep learning does not replace LKH, but helps it by producing better candidate sets and node penalties. The project confirms several important points: hybrid methods can be more powerful than pure hand-crafted heuristics, candidate-set quality is crucial for LKH performance, the graph representation and multi-task learning objective are meaningful design choices, and robust benchmark parsing matters a lot in practice.

At the same time, the project also shows how difficult it is to reproduce a research paper exactly. Differences in the training labels, remaining benchmark limitations, and engineering simplifications all affect the final behavior.

7 Conclusion

This project studied TSP through two strategies: classical LKH as a baseline and a hybrid NeuroLKH-style method as the main implementation. The hybrid method uses a Sparse Graph Network to predict edge scores and node penalties, then integrates these predictions into LKH.

The design choices were motivated by the original NeuroLKH paper and by practical considerations such as computational efficiency, modularity, and readability. The sparse directed graph representation, the multi-task training loss, the clearer TSPLIB benchmarking pipeline, and the modular separation of the codebase were all appropriate for the goals of the project.

The experiments showed that the implementation can be trained successfully and can be run on benchmark instances. In addition, the implementation differs from the paper in some important ways, most notably by using LKH rather than Concorde to generate training labels.

In conclusion, the project is a meaningful and functional implementation inspired by NeuroLKH. It captures the main algorithmic idea of combining deep learning with a strong classical heuristic and provides a clear basis for further improvements, such as stronger dataset generation and a more faithful reproduction of the original paper.

References

[1] Liang Xin, Wen Song, Zhiguang Cao, and Jie Zhang. *NeuroLKH: Combining Deep Learning Model with Lin-Kernighan-Helsgaun Heuristic for Solving the Traveling Salesman Problem.* In Advances in Neural Information Processing Systems (NeurIPS), 2021.

[2] Keld Helsgaun. An Effective Implementation of the Lin-Kernighan Traveling Salesman Heuristic. *European Journal of Operational Research*, 126(1):106–130, 2000.

[3] Keld Helsgaun. General k-opt Submoves for the Lin-Kernighan TSP Heuristic. *Mathematical Programming Computation*, 1(2–3):119–163, 2009.